Московский государственный университет имени М.В. Ломоносова
Механико-математический факультет

Численное решение задачи Стокса в единичном квадрате методом конечных разностей на смещенных сетках (MAC-метод)

Выполнил:
Филатов Никита
Группа 405

Москва, 2026

Содержание

1 Постановка задачи **2**

2 Аппроксимация системы уравнений центральными разностями на MAC-сетке **2**

2.1 Исходная система уравнений 2

3 Матричная форма и итерационный метод **5**

3.1 Блочная структура СЛАУ 5

3.2 Свойства матриц 6

3.2.1 Матрицы A_u и A_v 6

3.2.2 Матрицы B_1 и B_2 6

3.3 Итерационный метод Удзавы 6

3.3.1 Алгоритм 6

3.3.2 Выбор параметра τ 7

3.4 Критерий остановки 7

3.5 Замечание о выполнении LBB-условия 7

4 Теоретические основы **7**

4.1 Слабая постановка задачи Стокса 7

4.2 Конечноэлементная дискретизация 8

4.3 LBB-условие 8

5 Тесты. **9**

5.1 Тест 1. 9

5.2 Тест 2. 10

5.3 Тест 3. 11

5.4 Тест 4. 12

6 Физическая интерпретация задачи Стокса **13**

6.1 Поле скорости $\mathbf{u} = (u, v)^T$ 13

6.2 Давление $p(x, y)$ 13

6.3 Баланс сил: уравнение движения 14

6.4 Условие несжимаемости: уравнение неразрывности 14

7 Вывод **14**

8 Список литературы **14**

9 Листинг программы **15**

9.1 Класс MAC-сетки 15

9.2 Построение матриц 16

9.3 Метод Удзавы 16

1 Постановка задачи

Для задачи Стокса в единичном квадрате:

$$\begin{cases} -\Delta \mathbf{u} + \nabla p = \mathbf{f}, \\ \operatorname{div} \mathbf{u} = 0, \end{cases} \tag{1}$$

с условиями:

$$(p, \mathbf{1}) = 0, \qquad \mathbf{u}|_{\partial\Omega} = \mathbf{0}, \tag{2}$$

требуется:

1. построить разностную схему;
2. решить полученную систему линейных алгебраических уравнений (СЛАУ) при помощи итерационного метода.

Здесь $\mathbf{u} = (u(x,y), v(x,y))^T$ – вектор скорости, $p = p(x,y)$ – давление, $\mathbf{f} = (f_1(x,y), f_2(x,y))^T$ – заданная правая часть (внешние силы). Условие $(p, 1) = 0$ является условием нормировки, которое обеспечивает единственность давления (давление определяется с точностью до аддитивной константы).

Особенностью данной постановки является то, что граничные условия задаются только для скорости; для давления дополнительные условия на границе не требуются.

2 Аппроксимация системы уравнений центральными разностями на MAC-сетке

2.1 Исходная система уравнений

В скалярной форме система (1) имеет вид:

$$\begin{cases} -(u_{xx} + u_{yy}) + p_x = f_1, \\ -(v_{xx} + v_{yy}) + p_y = f_2, \\ u_x + v_y = 0, \end{cases} \qquad \begin{aligned} &u|_\Gamma = 0,\ v|_\Gamma = 0, \\ &(p, \mathbf{1}) = 0. \end{aligned} \tag{3}$$

Особенностью данной постановки является то, что граничные условия задаются только для скорости; для давления дополнительные условия на границе не требуются.

2.2 Выбор сетки: MAC-метод

Для выполнения LBB-условия используем MAC-схему (Marker-And-Cell), предложенную в [1] на смещённых сетках. Введём равномерную сетку с шагом $h = 1/N$, где N – количество ячеек по каждому направлению. Расположение узлов выбираем следующим образом:

- Давление p задаётся в центрах ячеек: $((i + 1/2)h,\ (j + 1/2)h), \quad i, j = 0, \ldots, N-1$;
- Горизонтальная компонента скорости u задаётся на вертикальных гранях: $((i+1/2)h,\ jh)$, $i = 0, \ldots, N-1, \quad j = 0, \ldots, N$;
- Вертикальная компонента скорости v задаётся на горизонтальных гранях: $(ih,\ (j + 1/2)h), \quad i = 0, \ldots, N, \quad j = 0, \ldots, N-1$.

2.3 Аппроксимация уравнений центральными разностями

Аппроксимация уравнения для u

Уравнение для горизонтальной компоненты скорости u рассматривается в узлах $((i+1/2)h,\ jh)$. Введём обозначения:

$$u_{i,j} = u\big((i+1/2)h,\ jh\big), \quad i = 0, \dots, N-1,\ j = 0, \dots, N. \tag{4}$$

Лапласиан:

$$\Delta u \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}. \tag{4}$$

Градиент давления:

$$p_x \approx \frac{p_{i+1,j} - p_{i,j}}{h}, \tag{5}$$

где $p_{i,j} = p\big((i+1/2)h,\ (j+1/2)h\big), \quad i, j = 0, \dots, N-1.$

Таким образом, разностное уравнение для u:

$$-\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} - \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2} + \frac{p_{i+1,j} - p_{i,j}}{h} = f^{(u)}_{1\,i,j}. \tag{6}$$

Аппроксимация уравнения для v

Уравнение для вертикальной компоненты скорости v рассматривается в узлах $(ih,\ (j+1/2)h)$. Введём обозначения:

$$v_{i,j} = v\big(ih,\ (j+1/2)h\big), \quad i = 0, \dots, N,\ j = 0, \dots, N-1. \tag{5}$$

Лапласиан:

$$\Delta v \approx \frac{v_{i+1,j} - 2v_{i,j} + v_{i-1,j}}{h^2} + \frac{v_{i,j+1} - 2v_{i,j} + v_{i,j-1}}{h^2}. \tag{7}$$

Градиент давления:

$$p_y \approx \frac{p_{i,j+1} - p_{i,j}}{h}. \tag{8}$$

Разностное уравнение для v:

$$-\frac{v_{i+1,j} - 2v_{i,j} + v_{i-1,j}}{h^2} - \frac{v_{i,j+1} - 2v_{i,j} + v_{i,j-1}}{h^2} + \frac{p_{i,j+1} - p_{i,j}}{h} = f^{(v)}_{2\,i,j}. \tag{9}$$

Аппроксимация уравнения неразрывности

Уравнение неразрывности $u_x + v_y = 0$ аппроксимируется в центрах ячеек $((i+1/2)h,\ (j+1/2)h)$:

$$\frac{u_{i,j+1} - u_{i,j}}{h} + \frac{v_{i+1,j} - v_{i,j}}{h} = 0. \tag{10}$$

Граничные условия

Граничные условия для скорости задаются на границе области Γ:

- На левой границе ($x = 0$): $u_{-1,j} = 0$ (фиктивный узел), $v_{0,j} = 0$;
- На правой границе ($x = 1$): $u_{N,j} = 0$, $v_{N,j} = 0$;
- На нижней границе ($y = 0$): $u_{i,-1} = 0$, $v_{i,0} = 0$;

- На верхней границе ($y = 1$): $u_{i,N} = 0$, $v_{i,N-1} = 0$.

В терминах физических узлов:

$$\begin{aligned} &u_{0,j} = 0,\ v_{0,j} = 0, && \text{при } x = 0,\\ &u_{N-1,j} = 0,\ v_{N,j} = 0, && \text{при } x = 1,\\ &u_{i,0} = 0,\ v_{i,0} = 0, && \text{при } y = 0,\\ &u_{i,N-1} = 0,\ v_{i,N-1} = 0, && \text{при } y = 1. \end{aligned} \tag{6}$$

Подсчитаем количество граничных условий:

- Для u: условия $u_{0,j} = 0$ ($j = 0, \dots, N$) дают $N+1$ уравнений, $u_{N-1,j} = 0$ ($j = 0, \dots, N$) дают $N + 1$ уравнений, $u_{i,0} = 0$ ($i = 0, \dots, N - 1$) дают N уравнений, $u_{i,N-1} = 0$ ($i = 0, \dots, N-1$) дают N уравнений. Угловые точки учтены дважды, поэтому количество уникальных уравнений для u равно $4N - 2$.

- Для v: условия $v_{0,j} = 0$ ($j = 0, \dots, N-1$) дают N уравнений, $v_{N,j} = 0$ ($j = 0, \dots, N-1$) дают N уравнений, $v_{i,0} = 0$ ($i = 0, \dots, N$) дают $N + 1$ уравнений, $v_{i,N-1} = 0$ ($i = 0, \dots, N$) дают $N+1$ уравнений. Угловые точки учтены дважды, поэтому количество уникальных уравнений для v равно $4N - 2$.

Таким образом, общее количество граничных условий для скорости составляет $8N - 4$. Условие нормировки давления:

$$\sum_{i=0}^{N-1} \sum_{j=0}^{N-1} p_{i,j} = 0. \tag{11}$$

Полученная система разностных уравнений

В результате аппроксимации центральными разностями на MAC-сетке получена следующая система линейных алгебраических уравнений:

$$\begin{cases} -\dfrac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} - \dfrac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2} + \dfrac{p_{i+1,j} - p_{i,j}}{h} = f^{(u)}_{1\,i,j}, & \begin{matrix} i = 1, \dots, N-2 \\ j = 1, \dots, N-1 \end{matrix} \\ -\dfrac{v_{i+1,j} - 2v_{i,j} + v_{i-1,j}}{h^2} - \dfrac{v_{i,j+1} - 2v_{i,j} + v_{i,j-1}}{h^2} + \dfrac{p_{i,j+1} - p_{i,j}}{h} = f^{(v)}_{2\,i,j}, & \begin{matrix} i = 1, \dots, N-1 \\ j = 1, \dots, N-2 \end{matrix} \\ \dfrac{u_{i+1,j} - u_{i,j}}{h} + \dfrac{v_{i,j+1} - v_{i,j}}{h} = 0, & \begin{matrix} i = 0, \dots, N-1 \\ j = 0, \dots, N-1 \end{matrix} \end{cases}$$

количество неизвестных: $3N^2 + 2N$;
количество уравнений: $3N^2 + 2N$.
Данная система была получена в [3]

Замечание об индексации

В данной индексации:

- $u_{i,j}$ определена для $i = 0, \dots, N - 1$, $j = 0, \dots, N$;

- $v_{i,j}$ определена для $i = 0, \dots, N$, $j = 0, \dots, N - 1$;

- $p_{i,j}$ определена для $i, j = 0, \ldots, N-1$.

При этом физические координаты узлов:

- $u_{i,j}$ в точке $((i+1/2)h,\ jh)$;
- $v_{i,j}$ в точке $(ih,\ (j+1/2)h)$;
- $p_{i,j}$ в точке $((i+1/2)h,\ (j+1/2)h)$.

Такая индексация позволяет использовать целые индексы, где сдвиг на 1/2 уже учтён в определении сеточных функций.

Свойства разностной схемы

Построенная разностная схема обладает следующими свойствами:

- Второй порядок аппроксимации $O(h)$;
- Выполнение дискретного аналога условия LBB.

3 Матричная форма и итерационный метод

После аппроксимации задачи Стокса центральными разностями на MAC-сетке получена система линейных алгебраических уравнений (СЛАУ). Запишем её в матричном виде, удобном для численного решения.

3.1 Блочная структура СЛАУ

Введём векторы неизвестных:

$$\mathbf{u} \in \mathbb{R}^{N_u}, \quad N_u = N(N+1)$$
$$\mathbf{v} \in \mathbb{R}^{N_v}, \quad N_v = (N+1)N$$
$$\mathbf{p} \in \mathbb{R}^{N_p}, \quad N_p = N^2$$

где N — количество ячеек сетки по каждому направлению (шаг $h = 1/N$).

Это точно те же величины, которые подробно описаны раньше в разделах 2.2–2.3:

$\mathbf{u}$ — вектор-столбец, составленный из всех значений $u_{i,j}$ (горизонтальная компонента скорости на вертикальных гранях ячеек MAC-сетки); $\mathbf{v}$ — вектор-столбец из всех $v_{i,j}$ (вертикальная компонента скорости на горизонтальных гранях); $\mathbf{p}$ — вектор-столбец из всех $p_{i,j}$ (давление в центрах ячеек).

Тогда дискретная система принимает вид:

$$\begin{pmatrix} A_u & 0 & B_1 \\ 0 & A_v & B_2 \\ B_1^T & B_2^T & 0 \end{pmatrix} \begin{pmatrix} \mathbf{u} \\ \mathbf{v} \\ \mathbf{p} \end{pmatrix} = \begin{pmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{0} \end{pmatrix}, \qquad (1)$$

где:

- A_u и A_v — дискретные аналоги оператора $-\Delta$ для горизонтальной и вертикальной компонент скорости соответственно;
- B_1 и B_2 — дискретные операторы градиента давления, действующие на u и v;
- B_1^T, B_2^T — дискретные операторы дивергенции;
- $\mathbf{f}_1, \mathbf{f}_2$ — правые части, полученные из f_1, f_2.

3.2 Свойства матриц

3.2.1 Матрицы A_u и A_v

Каждая из матриц A_u, A_v является симметричной, положительно определённой, пятидиагональной. Для внутреннего узла (i, j):

$$(-\Delta u)_{i,j} \approx \frac{4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1}}{h^2}.$$

На границе выполняются условия Дирихле ($u = 0, v = 0$), поэтому соответствующие строки матрицы заменяются на единичные.

3.2.2 Матрицы B_1 и B_2

Дискретный градиент давления аппроксимируется на полушаге:

$$\left.\frac{\partial p}{\partial x}\right|_{i+1/2,j} \approx \frac{p_{i+1,j} - p_{i,j}}{h}, \qquad \left.\frac{\partial p}{\partial y}\right|_{i,j+1/2} \approx \frac{p_{i,j+1} - p_{i,j}}{h}.$$

Таким образом, каждая строка B_1 содержит $+\frac{1}{h}$ и $-\frac{1}{h}$ для соответствующих давлений, а остальные элементы равны нулю. Аналогично устроена B_2.

3.3 Итерационный метод Удзавы

Для решения системы (1) применим метод Удзавы (Uzawa) — итерационный алгоритм, основанный на фиксации давления и последующей корректировке по невязке уравнения неразрывности. Этот алгоритм был предложен в [2]

3.3.1 Алгоритм

1. **Начальное приближение:**

$$\mathbf{p}^0 = \mathbf{0}, \quad \mathbf{u}^0 = \mathbf{0}, \quad \mathbf{v}^0 = \mathbf{0}.$$

2. **Выбор параметра:** $0 < \tau < \frac{2}{\lambda_{\max}(S)}$, где $S = B^T A^{-1} B$ — матрица Шура.

3. **Итерации** $k = 0, 1, 2, \ldots$:

   а) **Шаг скоростей** (при фиксированном давлении):

$$A_u \mathbf{u}^{k+1} = \mathbf{f}_1 - B_1 \mathbf{p}^k,$$
$$A_v \mathbf{v}^{k+1} = \mathbf{f}_2 - B_2 \mathbf{p}^k.$$

   Обе системы решаются независимо методом сопряжённых градиентов.

   б) **Невязка (дивергенция):**

$$\mathbf{r}^{k+1} = B_1^T \mathbf{u}^{k+1} + B_2^T \mathbf{v}^{k+1}.$$

   в) **Обновление давления:**

$$\mathbf{p}^{k+1} = \mathbf{p}^k + \tau \mathbf{r}^{k+1}.$$

   г) **Проекция на нулевое среднее** (обеспечивает единственность давления):

$$p_{i,j}^{k+1} \leftarrow p_{i,j}^{k+1} - \frac{1}{N^2} \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} p_{m,n}^{k+1}.$$

   д) **Проверка сходимости:**

$$\|\mathbf{r}^{k+1}\|_2 < \varepsilon,$$

   где ε — заданная точность (например, $\varepsilon = 10^{-8}$).

3.3.2 Выбор параметра τ

Матрица Шура $S = B_1^T A_u^{-1} B_1 + B_2^T A_v^{-1} B_2$ симметрична и положительно определена. Метод Удзавы сходится при условии:

$$0 < \tau < \frac{2}{\lambda_{\max}(S)}.$$

Оптимальный параметр, минимизирующий спектральный радиус итерационной матрицы:

$$\tau_{\text{opt}} = \frac{2}{\lambda_{\min}(S) + \lambda_{\max}(S)}.$$

3.4 Критерий остановки

В качестве основного критерия остановки используется относительная норма невязки уравнения неразрывности:

$$\frac{\|\mathbf{r}^{k+1}\|_2}{\|\mathbf{r}^0\|_2 + \delta} < \varepsilon,$$

где δ — малая константа, предотвращающая деление на ноль. Дополнительно можно контролировать изменение давления:

$$\|\mathbf{p}^{k+1} - \mathbf{p}^k\|_2 < \varepsilon_p.$$

3.5 Замечание о выполнении LBB-условия

Использование MAC-сетки со смещёнными узлами гарантирует выполнение дискретного аналога LBB-условия (Ладыженской–Бабушки–Брецци). Это обеспечивает устойчивость численного решения и отсутствие паразитных осцилляций давления. Матрица Шура S при этом является хорошо обусловленной, что позволяет методу Удзавы сходиться за приемлемое число итераций.

4 Теоретические основы

Данная теория была взята из [1]

4.1 Слабая постановка задачи Стокса

Для построения численных методов удобно перейти к слабой (вариационной) постановке. Введём функциональные пространства:

$$\begin{aligned}
L_2(\Omega) &= \left\{ f : \int_\Omega f^2(x,y)\,dxdy < \infty \right\}, \\
P &= \left\{ p \in L_2(\Omega) : \int_\Omega p\,dxdy = 0 \right\}, \\
\mathbf{U} &= \mathring{W}_2^1(\Omega) \times \mathring{W}_2^1(\Omega), \\
\mathring{W}_2^1(\Omega) &= \{ u : u, u_x, u_y \in L_2(\Omega),\ u|_\Gamma = 0 \}.
\end{aligned}$$

Умножим первое уравнение (1) на пробную функцию $\mathbf{v} \in \mathbf{U}$, второе – на пробную функцию $q \in P$ и проинтегрируем по Ω. После интегрирования по частям с учётом граничных условий получаем следующую слабую постановку:

$$\begin{cases} (\operatorname{grad}\mathbf{u}, \operatorname{grad}\mathbf{v}) - (p, \operatorname{div}\mathbf{v}) = (\mathbf{f}, \mathbf{v}) \quad \forall \mathbf{v} \in \mathbf{U}, \\ -(q, \operatorname{div}\mathbf{u}) = 0 \quad \forall q \in P, \end{cases} \tag{7}$$

где круглые скобки обозначают скалярное произведение в $L_2(\Omega)$:

$$(a, b) = \int_\Omega a(x, y) b(x, y)\, dxdy,$$

а для векторных функций:

$$(\operatorname{grad}\mathbf{u}, \operatorname{grad}\mathbf{v}) = \sum_{i=1}^{2} \left[\left(\frac{\partial u_i}{\partial x}, \frac{\partial v_i}{\partial x} \right) + \left(\frac{\partial u_i}{\partial y}, \frac{\partial v_i}{\partial y} \right) \right].$$

4.2 Конечноэлементная дискретизация

Для дискретизации задачи (7) необходимо ввести конечномерные подпространства $\mathbf{U}_h \subset \mathbf{U}$ и $P_h \subset P$, задаваемые в виде линейных оболочек наборов базисных функций:

$$\mathbf{U}_h = \left(\operatorname{span}\{\varphi_1, \varphi_2, \ldots, \varphi_{N_u/2}\}\right)^2, \quad P_h = \operatorname{span}\{\psi_1, \psi_2, \ldots, \psi_{N_p}\},$$

так что приближенное решение представимо в виде:

$$\mathbf{u}_h = \sum_{i=1}^{N_u/2} \mathbf{u}_i \varphi_i(x, y), \quad p_h = \sum_{j=1}^{N_p} p_j \psi_j(x, y).$$

Отметим, что в представлении скорости коэффициент $\mathbf{u}_i$ является двухмерным вектором $(v_i, w_i)^T$. Это приводит к конечномерной задаче: найти $\mathbf{u}_h \in \mathbf{U}_h$ и $p_h \in P_h$ такие, что

$$\begin{cases} (\operatorname{grad}\mathbf{u}_h, \operatorname{grad}\mathbf{v}_h) - (p_h, \operatorname{div}\mathbf{v}_h) = (\mathbf{f}, \mathbf{v}_h) \quad \forall \mathbf{v}_h \in \mathbf{U}_h, \\ -(q_h, \operatorname{div}\mathbf{u}_h) = 0 \quad \forall q_h \in P_h. \end{cases} \tag{8}$$

4.3 LBB-условие

При дискретизации задачи Стокса возникает фундаментальное требование согласованности аппроксимаций скорости и давления – условие Ладыженской – Бабушки – Брецци (LBB-условие). Пусть $\mathbf{U}_h \subset \mathbf{U}$ и $P_h \subset P$ – конечномерные подпространства, аппроксимирующие скорость и давление соответственно. LBB-условие формулируется следующим образом: существует постоянная $\delta > 0$, не зависящая от параметра сетки h, такая что

$$\sup_{\mathbf{u}_h \in \mathbf{U}_h} \frac{(p_h, \operatorname{div}\mathbf{u}_h)}{(\operatorname{grad}\mathbf{u}_h, \operatorname{grad}\mathbf{u}_h)^{1/2}} \geq \delta \|p_h\|_{L_2} \quad \forall p_h \in P_h. \tag{9}$$

При выполнении этого условия гарантируется существование, единственность и устойчивость решения дискретной задачи. Нарушение LBB-условия приводит к нефизичным осцилляциям давления (так называемое “шахматное” давление) или неединственности решения.

5 Тесты.

5.1 Тест 1.

Для численного метода были выбраны следующие функции, задающие точное решение задачи Стокса в единичном квадрате:

$$u(x,y) = 20x^2(1-x)^2y(1-y)(1-2y), \tag{10}$$
$$v(x,y) = -20x(1-x)(1-2x)y^2(1-y)^2, \tag{11}$$
$$p(x,y) = \left(x-\frac{1}{2}\right)\left(y-\frac{1}{2}\right). \tag{12}$$

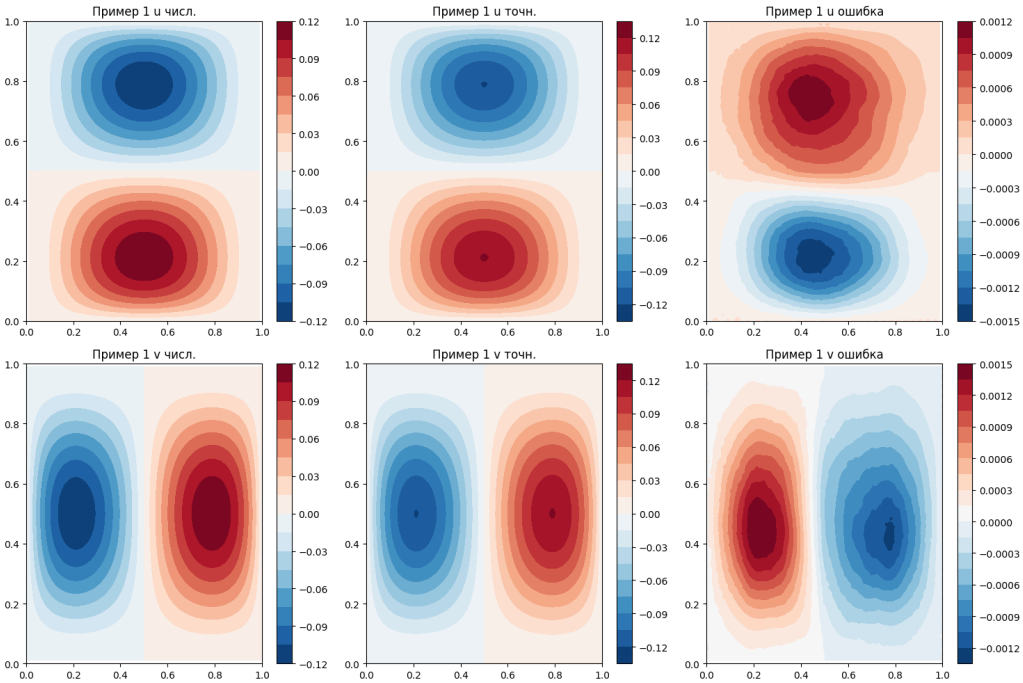

Рис. 1: скорости u, v (N=64)

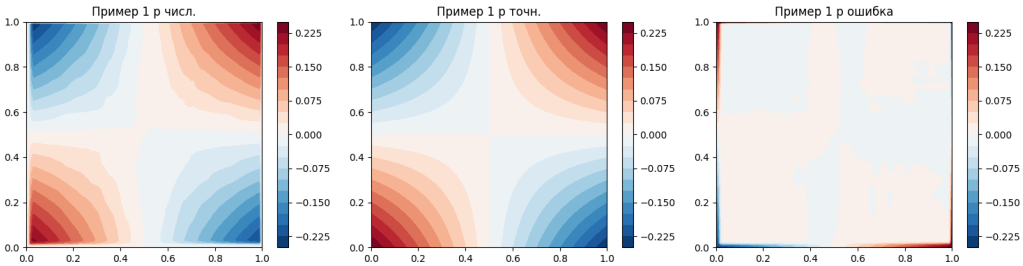

Рис. 2: давление p (N=64)

Полученные значения погрешностей в L_2-норме:

Величина	L_2-погрешность
u	5.764039×10^{-4}
v	5.851602×10^{-4}
p	2.528322×10^{-2}

Таблица 1: Погрешности численного решения

5.2 Тест 2.

Для численного метода были выбраны следующие функции, задающие точное решение задачи Стокса в единичном квадрате:

$$\begin{cases} u(x,y) = \sin^2(\pi x)\sin(2\pi y), \\ v(x,y) = -\sin(2\pi x)\sin^2(\pi y), \\ p(x,y) = \cos(2\pi x)\cos(2\pi y). \end{cases} \tag{13}$$

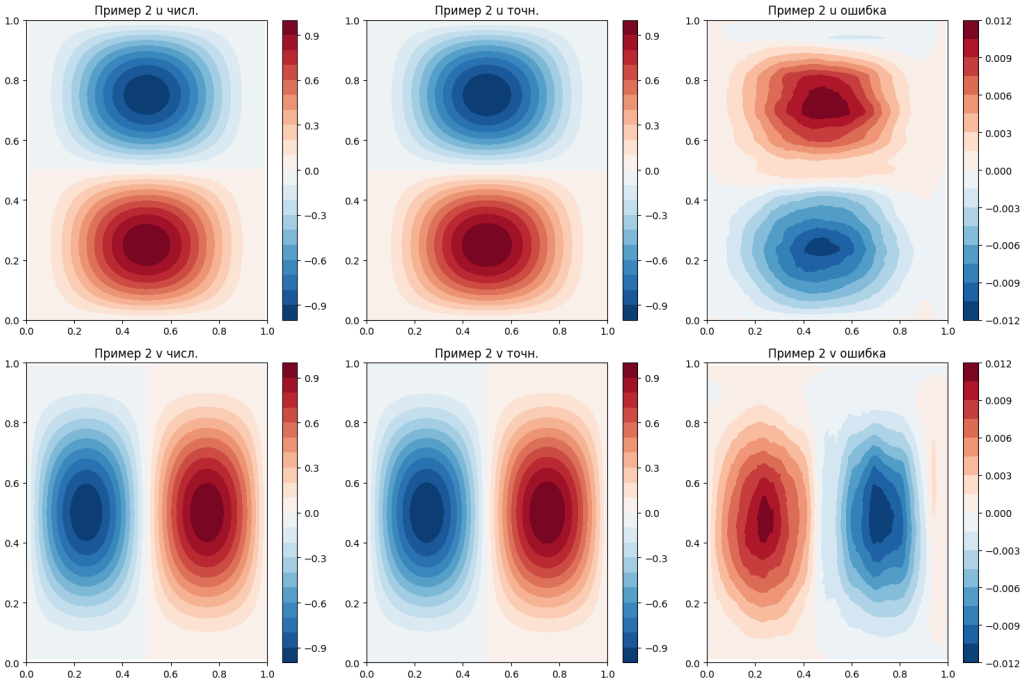

Рис. 3: скорости u, v (N=64)

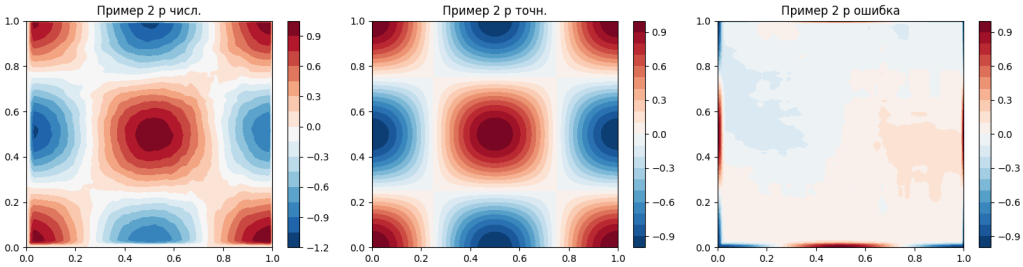

Рис. 4: давление p (N=64)

Полученные значения погрешностей в L_2-норме:

Величина	L_2-погрешность
u	4.482819×10^{-3}
v	4.482819×10^{-3}
p	1.429252×10^{-1}

Таблица 2: Погрешности численного решения

5.3 Тест 3.

Для численного метода были выбраны следующие функции, задающие точное решение задачи Стокса в единичном квадрате:

$$\begin{cases} u(x,y) = \sin^2(2\pi x)\sin(2\pi y)\cos(2\pi y), \\ v(x,y) = -\sin(2\pi x)\cos(2\pi x)\sin^2(2\pi y), \\ p(x,y) = \cos(2\pi x)\cos(2\pi y). \end{cases} \tag{14}$$

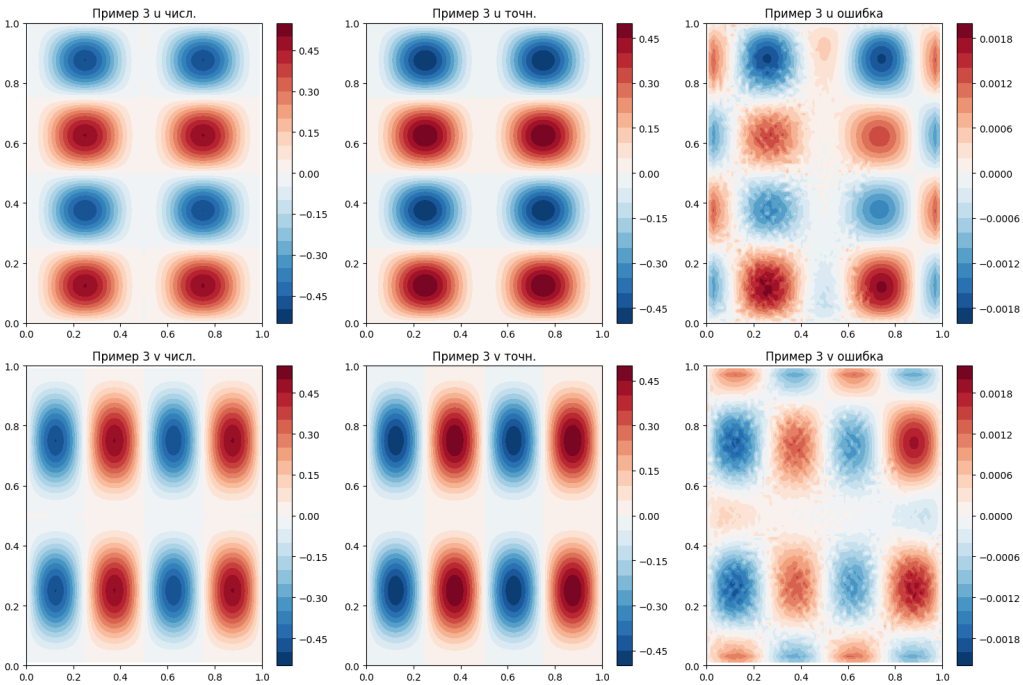

Рис. 5: скорости u, v (N=64)

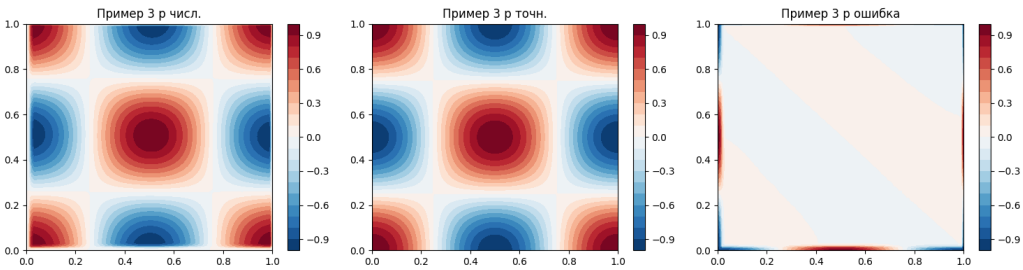

Рис. 6: давление p (N=64)

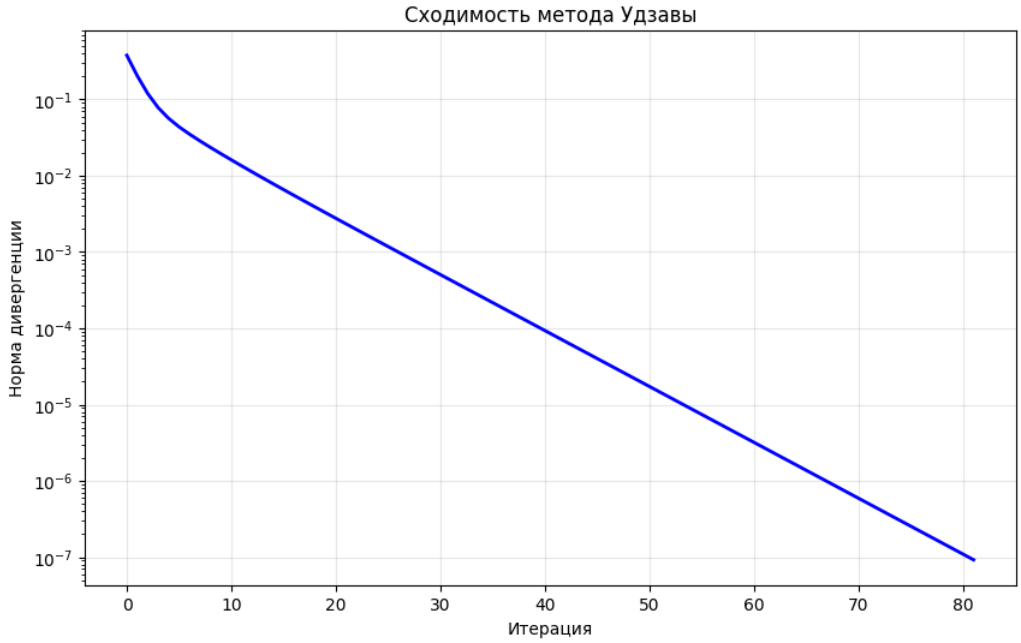

Рис. 7: сходимость (N=64)

Полученные значения погрешностей в L_2-норме:

Величина	L_2-погрешность
u	7.086394×10^{-4}
v	7.086400×10^{-4}
p	1.286607×10^{-1}

Таблица 3: Погрешности численного решения

5.4 Тест 4.

Для 3 теста было проведено продолжение по параметру с правой частью, равной: $\lambda f(x)$, где $\lambda = [-10.0, -1.0, -0.5, 0.5, 1.0, 9.0, 9.5, 9.7, 10.0]$.

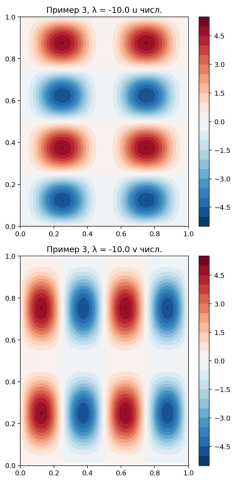

Рис. 8: $\lambda = -10$

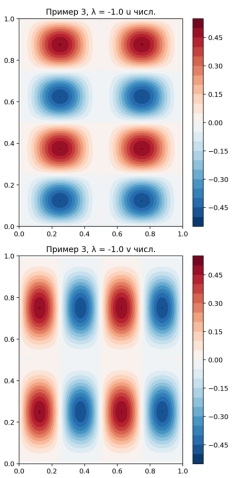

Рис. 9: $\lambda = -1$

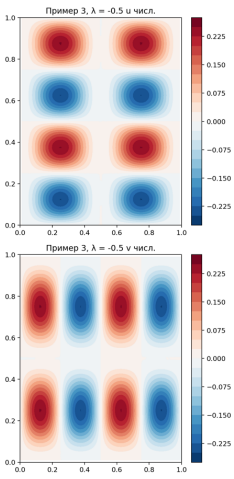

Рис. 10: $\lambda = -0.5$

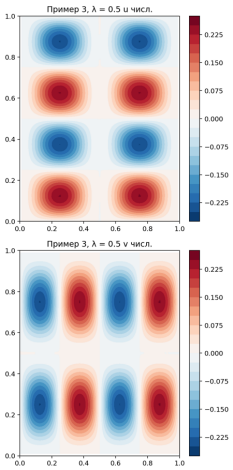

Рис. 11: $\lambda = 0.5$

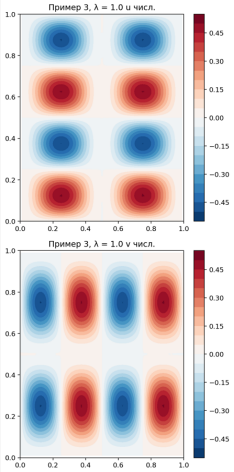

Рис. 12: $\lambda = 1$

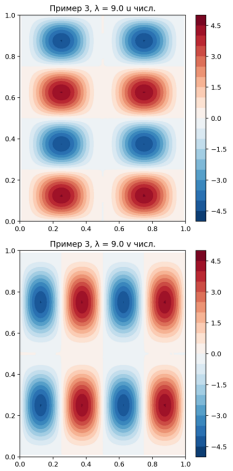

Рис. 13: $\lambda = 9$

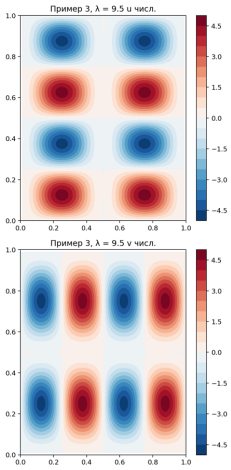

Рис. 14: $\lambda = 9.5$

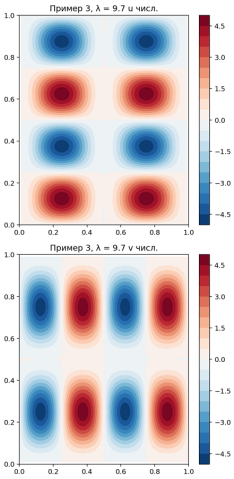

Рис. 15: $\lambda = 9.7$

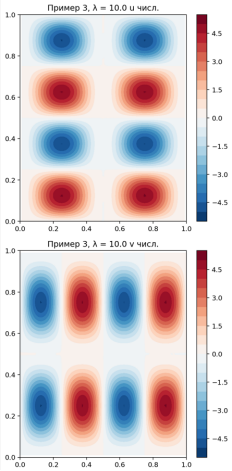

Рис. 16: $\lambda = 10$

6 Физическая интерпретация задачи Стокса

Задача Стокса описывает стационарное течение медленной вязкой несжимаемой жидкости. Рассмотрим физический смысл каждого компонента системы.

6.1 Поле скорости $\mathbf{u} = (u, v)^T$

Вектор скорости $\mathbf{u}(x, y)$ характеризует движение жидких частиц:

- $u(x, y)$ — горизонтальная компонента скорости (проекция на ось x);
- $v(x, y)$ — вертикальная компонента скорости (проекция на ось y).

В каждой точке области Ω вектор скорости указывает направление и величину переноса массы жидкости. На границе Γ выполняется условие прилипания $\mathbf{u}|_\Gamma = \mathbf{0}$, что означает:

- жидкость не проникает через стенки;
- жидкость не скользит вдоль стенок (вязкое трение останавливает её у границы).

6.2 Давление $p(x, y)$

Давление p — это скалярное поле, представляющее собой нормальные напряжения (силы, действующие перпендикулярно площадке) внутри жидкости.

Физически давление:

- создает движущую силу: перепад давления ∇p заставляет жидкость течь;
- поддерживает условие несжимаемости, "подстраиваясь"так, чтобы дивергенция скорости равнялась нулю;
- определяется с точностью до константы — важно лишь изменение давления, а не его абсолютная величина.

Условие нормировки $\int_\Omega p\, dxdy = 0$ фиксирует эту константу, выбирая давление с нулевым средним.

6.3 Баланс сил: уравнение движения

Первое уравнение системы

$$-\Delta \mathbf{u} + \nabla p = \mathbf{f}$$

выражает баланс сил в каждой точке жидкости:

- $\mathbf{f}(x, y)$ — заданные внешние силы (например, сила тяжести, электромагнитные силы), действующие на жидкость;
- $-\Delta \mathbf{u}$ — вязкие силы (силы трения между слоями жидкости). Оператор Лапласа $\Delta \mathbf{u} = (u_{xx} + u_{yy}, v_{xx} + v_{yy})^T$ характеризует пространственные изменения скорости: чем больше неоднородность поля скорости, тем больше вязкое трение;
- ∇p — градиент давления (сила, вызванная перепадом давления).

Уравнение читается так: *силы вязкого трения уравновешиваются градиентом давления и внешними силами.*

6.4 Условие несжимаемости: уравнение неразрывности

Второе уравнение

$$\operatorname{div} \mathbf{u} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

выражает закон сохранения массы для несжимаемой жидкости.

Физический смысл: *сколько жидкости втекает в элементарный объем, столько и вытекает.* Другими словами, плотность жидкости остается постоянной — жидкость не сжимается и не расширяется.

7 Вывод

Проведённые численные эксперименты позволяют сделать следующие выводы:

1. Во всех трёх тестовых примерах численное решение устойчиво приближается к точному. Погрешности для компонент скорости u и v составляют порядка 10^{-3}–10^{-4}, что соответствует второму порядку аппроксимации схемы.
2. Для давления p наибольшая погрешность возникает на границе квадрата.
3. Для третьего теста было выполнено продолжение решения по параметру λ в правой части $\lambda \mathbf{f}(\mathbf{x})$ при $\lambda \in [-10.0,\ -1.0,\ -0.5,\ 0.5,\ 1.0,\ 9.0,\ 9.5,\ 9.7,\ 10.0]$. Метод сохраняет устойчивость во всём диапазоне изменения параметра, включая области вблизи критических значений. Также решения при малых изменениях правых частей не сильно отличаются.

8 Список литературы

Список литературы

[1] Чижонков Е.В. Численные методы. Курс лекций для механиков мехмата, 2012.

[2] Быченков Ю.В., Чижонков Е.В. Итерационные методы решения седловых задач. — М., 2009.

[3] Бахвалов Н.С., Корнев А.А., Чижонков Е.В. Численные методы. Решения задач и упражнения. — 2-е изд., испр. и доп. (электронное). — М.: Лаборатория знаний, 2016. — 355 с.

9 Листинг программы

Ниже представлен Python-код, реализующий метод MAC-сетки и алгоритм Удзавы.

9.1 Класс MAC-сетки

Листинг 1: Класс MAC-сетки

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import lil_matrix, csr_matrix
from scipy.sparse.linalg import cg
import time
from scipy.interpolate import griddata

class MACGrid:
    def __init__(self, N):
        self.N = N
        self.h = 1.0 / N

        self.nu_x = N
        self.nu_y = N + 1
        self.nv_x = N + 1
        self.nv_y = N
        self.np_x = N
        self.np_y = N

        self.n_u = self.nu_x * self.nu_y
        self.n_v = self.nv_x * self.nv_y
        self.n_p = self.np_x * self.np_y

        self.x_u = (np.arange(self.nu_x) + 0.5) * self.h
        self.y_u = np.arange(self.nu_y) * self.h
        self.x_v = np.arange(self.nv_x) * self.h
        self.y_v = (np.arange(self.nv_y) + 0.5) * self.h
        self.x_p = (np.arange(self.np_x) + 0.5) * self.h
        self.y_p = (np.arange(self.np_y) + 0.5) * self.h

    def idx_u(self, i, j):
        return i * self.nu_y + j

    def idx_v(self, i, j):
        return i * self.nv_y + j

    def idx_p(self, i, j):
        return i * self.np_y + j
```

9.2 Построение матриц

Листинг 2: Построение матриц операторов

```
def build_matrices(grid):
    h = grid.h
    h2_inv = 1.0 / (h * h)
    h_inv = 1.0 / h

    Au = lil_matrix((grid.n_u, grid.n_u))
    for i in range(grid.nu_x):
        for j in range(grid.nu_y):
            idx = grid.idx_u(i, j)
            if i == 0 or i == grid.nu_x - 1 or j == 0 or j == grid.nu_y - 1:
                Au[idx, idx] = 1.0
            else:
                Au[idx, idx] = 4.0 * h2_inv
                Au[idx, grid.idx_u(i-1, j)] = -h2_inv
                Au[idx, grid.idx_u(i+1, j)] = -h2_inv
                Au[idx, grid.idx_u(i, j-1)] = -h2_inv
                Au[idx, grid.idx_u(i, j+1)] = -h2_inv

    Av = lil_matrix((grid.n_v, grid.n_v))
    for i in range(grid.nv_x):
        for j in range(grid.nv_y):
            idx = grid.idx_v(i, j)
            if i == 0 or i == grid.nv_x - 1 or j == 0 or j == grid.nv_y - 1:
                Av[idx, idx] = 1.0
            else:
                Av[idx, idx] = 4.0 * h2_inv
                Av[idx, grid.idx_v(i-1, j)] = -h2_inv
                Av[idx, grid.idx_v(i+1, j)] = -h2_inv
                Av[idx, grid.idx_v(i, j-1)] = -h2_inv
                Av[idx, grid.idx_v(i, j+1)] = -h2_inv

    return csr_matrix(Au), csr_matrix(Av)
```

9.3 Метод Удзавы

Листинг 3: Метод Удзавы

```
class UzawaSolver:
    def __init__(self, grid, Au, Av, B1, B2, tau=None, use_cg=True):
        self.grid = grid
        self.Au = Au
        self.Av = Av
        self.B1 = B1
        self.B2 = B2
        self.use_cg = use_cg
        self.tau = tau if tau is not None else 0.5

```

```
    def solve(self, f1_vec, f2_vec, max_iter=5000, tol=1e-8):
        n_p = self.grid.n_p
        p = np.zeros(n_p)

        for it in range(max_iter):
            rhs_u = f1_vec - self.B1 @ p
            rhs_v = f2_vec - self.B2 @ p

            u = self.solve_u(rhs_u)
            v = self.solve_v(rhs_v)

            r = self.B1.T @ u + self.B2.T @ v
            p_new = p + self.tau * r
            p_new -= np.mean(p_new)

            r_norm = np.linalg.norm(r) * self.grid.h

            if r_norm < tol:
                break
            p = p_new

        return u, v, p
```